

ENTITY-RELATIONSHIP (ER) MODEL

The entity-relationship (ER) data model allows us to describe the data involved in a real-world enterprise in terms of objects and their relationships and is widely used to develop an initial database design.

Entity-relationship (ER) model is a popular high-level conceptual data model. This model and its variations are frequently used for the conceptual design of database applications.

E-R diagram can express the overall logical structure of a database graphically.

A database can be modeled as:

- a collection of entities,
- relationship among entities.

An **entity** is a “thing” or “object” in the real world that is distinguishable from all other objects. For example, each person in a university is an entity.

An entity has a set of properties, and the values for some set of properties must uniquely identify an entity. For instance, a person may have a person id property whose value uniquely identifies that person. Similarly, courses can be thought of as entities, and course id uniquely identifies a course entity in the university.

An entity is represented by a set of attributes. Attributes are descriptive properties possessed by each member of an entity set. Example: people have names and addresses

An **entity set** (collection of similar entities) is a set of entities of the same type that share the same properties, or attributes. An entity is described using a set of attributes. All entities in a given entity set have the same attributes; this is essentially what we mean by similar. The set of all people who are instructors at a given university, for example, can be defined as the entity set instructor. Similarly, the entity set student might represent the set of all students in the university.

Possible attributes of the instructor entity set are ID, name, dept name, and salary.

Possible attributes of the course entity set are course id, title, dept name, and credits.

For each attribute associated with an entity set, we must identify a domain of possible values. For example, the domain associated with the attribute name of Employees might be the set of 20-character strings.

A key is a minimal set of attributes whose values uniquely identify an entity in the set.

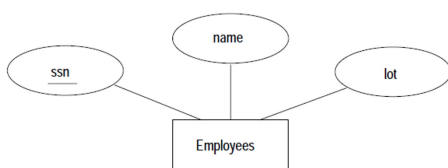


Figure The Employees Entity Set

An entity set is represented by a **rectangle**, and an attribute is represented by an **oval**.

The key is ssn.

Employees entity set use name, social security number (ssn), and parking lot (lot) as attributes.

Instructor

ID	name	dept name	Sal
-	-	-	-
-	-	-	-
-	-	-	-

*t₁
t₂
t₃*

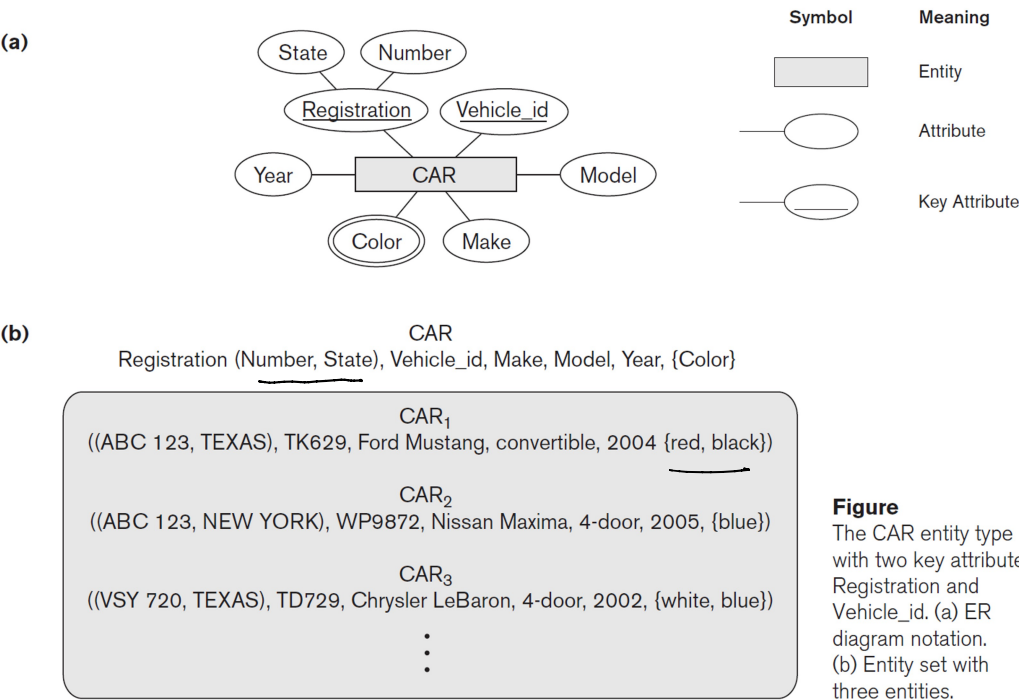
Employees

SSn	name	lot
-	-	-
-	-	-
-	-	-

Key Attributes of an Entity Type: An important constraint on the entities of an entity type is the key or uniqueness constraint on attributes. An entity type usually has one or more attributes whose values are distinct for each individual entity in the entity set. Such an attribute is called a key attribute, and its values can be used to identify each entity uniquely. In ER diagrammatic notation, each key attribute has its name underlined inside the oval.

Sometimes several attributes together form a key, meaning that the combination of the attribute values must be distinct for each entity. If a set of attributes possesses this property, the proper way to represent this in the ER model that we describe here is to define a composite attribute and designate it as a key attribute of the entity type. Notice that such a composite key must be minimal; that is, all component attributes must be included in the composite attribute to have the uniqueness property. Superfluous attributes must not be included in a key.

In our diagrammatic notation, if two attributes are underlined separately, then each is a key on its own.

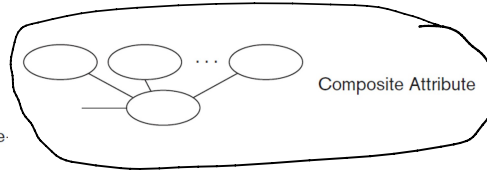
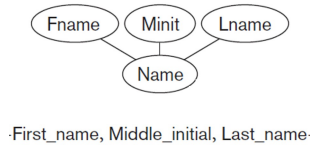
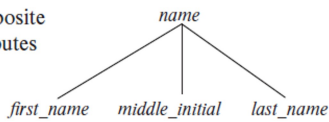


The Registration attribute is an example of a composite key formed from two simple component attributes, State and Number, neither of which is a key on its own.

An entity type may also have no key, in which case it is called a weak entity type.

Composite versus Simple (Atomic) Attributes: Composite attributes can be divided into smaller subparts, which represent more basic attributes with independent meanings. For example, the Address attribute can be subdivided into Street_address, City, State, and Zip. Attributes that are not divisible are called simple or atomic attributes

composite
attributes



component
attributes

If the composite attribute is referenced only as a whole, there is no need to subdivide it into component attributes. For example, if there is no need to refer to the individual components of an address (Zip Code, street, and so on), then the whole address can be designated as a simple attribute.

Single-Valued versus Multivalued Attributes: Most attributes have a single value for a particular entity; such attributes are called single-valued. For example, Age is a single-valued attribute of a person. In some cases an attribute can have a set of values for the same entity—for instance, a Colors attribute for a car, or a College_degrees attribute for a person. Cars with one color have a single value, whereas two-tone cars have two color values. Similarly, one person may not have any college degrees, another person may have one, and a third person may have two or more degrees; therefore, different people can have different numbers of values for the College_degrees attribute. Such attributes are called **multivalued**.



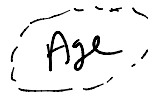
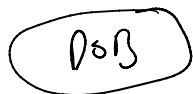
Multivalued Attribute

Stored versus Derived Attributes: In some cases, two (or more) attribute values are related—for example, the Age and Birth_date attributes of a person. For a particular person entity, the value of Age can be determined from the current (today's) date and the value of that person's Birth_date. The Age attribute is hence called a **derived attribute** and is said to be derivable from the Birth_date attribute, which is called a **stored attribute**.



Derived Attribute

DOB Age



RELATIONSHIPS AND RELATIONSHIP SETS



Relationship

Diamonds represent relationship sets

A **relationship** is an association among two or more entities. For example, we may have the relationship that Rahul works in the pharmacy department.

As with entities, we may wish to collect a set of similar relationships into a **relationship set**.

Relationship Set: Collection of similar relationships (a set of relationships of the same type).

A relationship set can be thought of as a set of n -tuples:

$$\{(e_1, \dots, e_n) \mid e_1 \in E_1, \dots, e_n \in E_n\}$$

Each n -tuple denotes a relationship involving n entities e_1 through e_n , where entity e_i is in entity set E_i .

In Figure, we show the relationship set Works_In, in which each relationship indicates a department in which an employee works. Note that several relationship sets might involve the same entity sets. For example, we could also have a Manages relationship set involving Employees and Departments.

A relationship set is represented in an E-R diagram by a diamond, which is linked via lines to a number of different entity sets (rectangles).

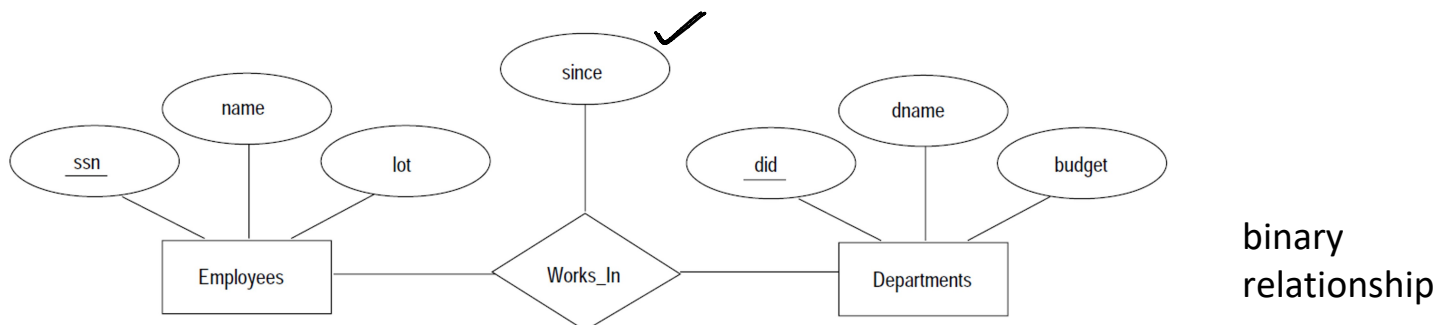
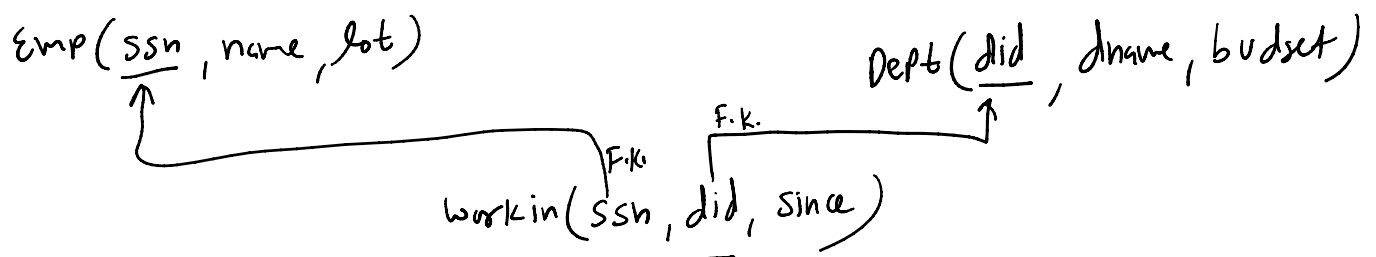
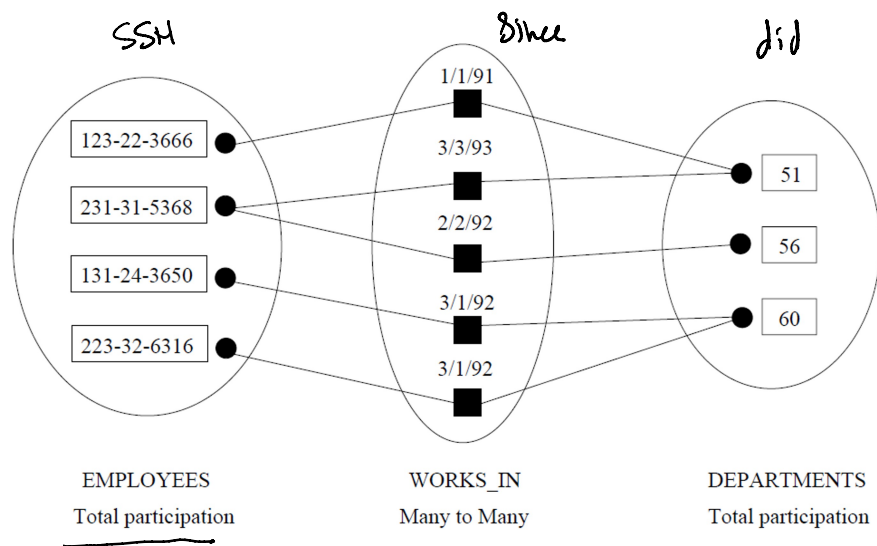


Figure The Works_In Relationship Set



A relationship can also have **descriptive attributes**. Descriptive attributes are used to record information about the relationship, rather than about any one of the participating entities; for example, we may wish to record that Rahul works in the pharmacy department as of January 1991. This information is captured in Figure by adding an attribute, since, to Works In.

An instance of a relationship set is a set of relationships. Intuitively, an instance can be thought of as a **'snapshot'** of the relationship set at some instant in time.



Works In relationship set, in which an employee is allowed to work in several departments and a department is allowed to have several employees, is said to be many-to-many.

Figure An Instance of the Works_In Relationship Set

As another example of an ER diagram, suppose that each department has offices in several locations and we want to record the locations at which each employee works. This relationship is **ternary** because we must record an association between an employee, a department, and a location. The ER diagram for this variant of Works In, which we call Works In2, is shown below

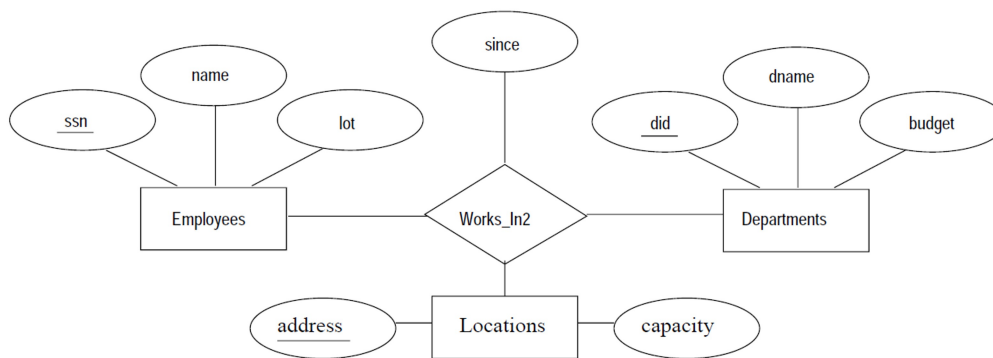


Figure A Ternary Relationship Set

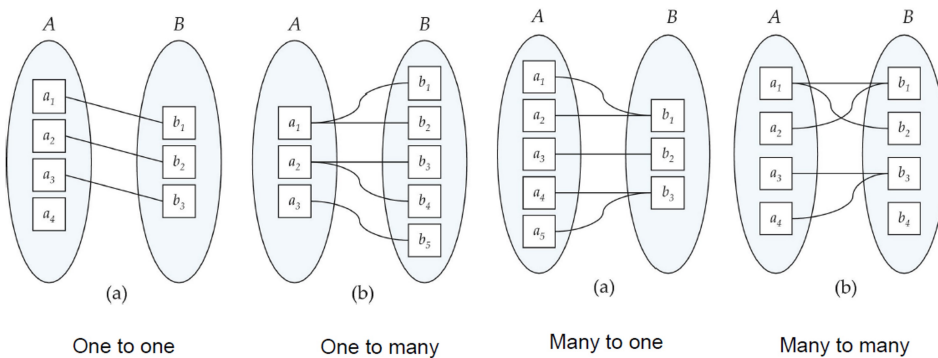
Mapping Cardinalities

Mapping cardinalities, or cardinality ratios, express the number of entities to which another entity can be associated via a relationship set.

Most useful in describing binary relationship sets.

For a binary relationship set the mapping cardinality must be one of the following types:

One to one, One to many, Many to one, Many to many

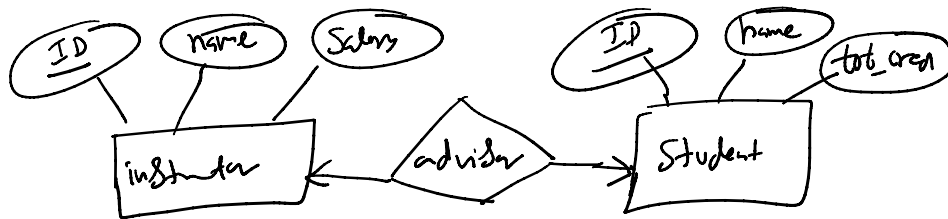
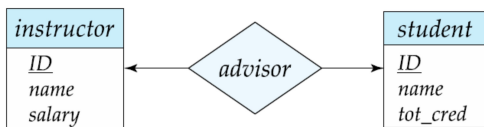


Note: Some elements in A and B may not be mapped to any elements in the other set

We express cardinality constraints by drawing either a directed line ($\rightarrow$), signifying "one," or an undirected line (—), signifying "many," between the relationship set and the entity set.

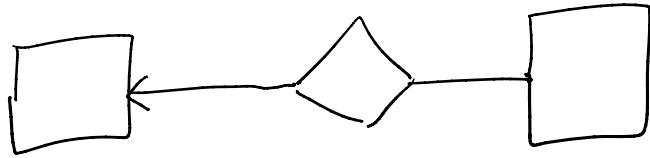
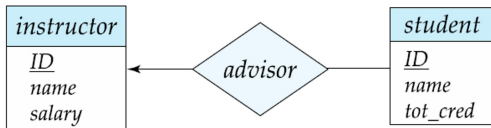
One to one relationship between an instructor and a student:

- An instructor is associated with at most one student via the relationship advisor
- A student is associated with at most one instructor via the relationship advisor



One to many relationship between an instructor and a student:

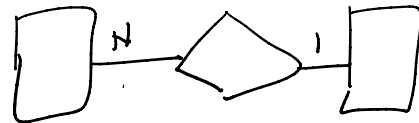
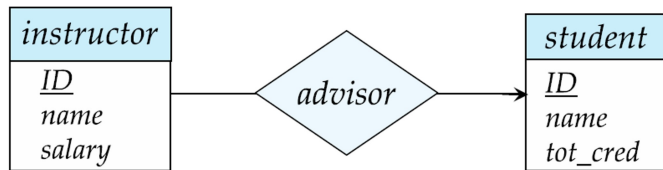
- an instructor is associated with several (including 0) students via advisor
- a student is associated with at most one instructor via advisor



Many-to-One Relationships

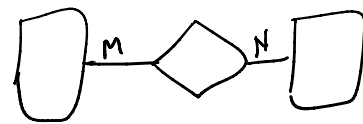
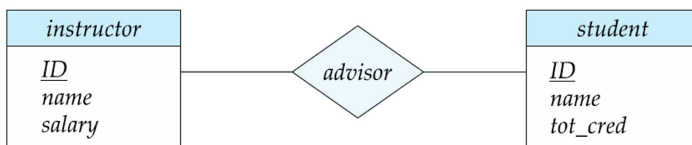
In a many-to-one relationship between an *instructor* and a *student*,

- an *instructor* is associated with at most one *student* via *advisor*,
- and a *student* is associated with several (including 0) *instructors* via *advisor*



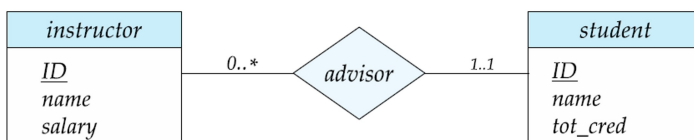
Many-to-Many Relationship

- An *instructor* is associated with several (possibly 0) *students* via *advisor*
- A *student* is associated with several (possibly 0) *instructors* via *advisor*

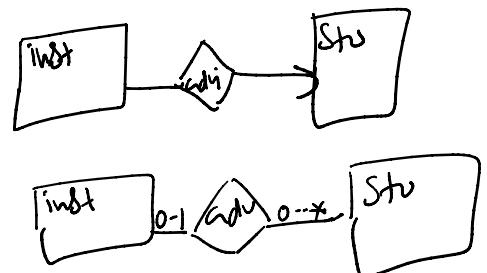


Notation for Expressing More Complex Constraints

- A line may have an associated minimum and maximum cardinality, shown in the form $l..h$, where l is the minimum and h the maximum cardinality
 - A minimum value of 1 indicates total participation.
 - A maximum value of 1 indicates that the entity participates in at most one relationship
 - A maximum value of * indicates no limit.
- Example



$l \dots h$ $0 \dots 1$
 $2 \dots *$



- Instructor can advise 0 or more students. A student must have 1 advisor; cannot have multiple advisors

The line between advisor and student has a cardinality constraint of 1..1, meaning the minimum and the maximum cardinality are both 1. That is, each student must have exactly one advisor. The limit 0.. * on the line between advisor and instructor indicates that an instructor can have zero or more students. Thus, the relationship advisor is one-to-many from instructor to student, and further the participation of student in advisor is total, implying that a student must have an advisor.

It is easy to misinterpret the 0.. * on the left edge and think that the relationship advisor is many-to-one from instructor to student—this is exactly the reverse of the correct interpretation.

If both edges have a maximum value of 1, the relationship is one-to-one. If we had specified a cardinality limit of 1.. * on the left edge, we would be saying that each instructor must advise at least one student.

Choice of Primary key for Binary Relationship

- Many-to-Many relationships. The preceding union of the primary keys is a minimal superkey and is chosen as the primary key.
- One-to-Many relationships. The primary key of the “Many” side is a minimal superkey and is used as the primary key.
- Many-to-one relationships. The primary key of the “Many” side is a minimal superkey and is used as the primary key.
- One-to-one relationships. The primary key of either one of the participating entity sets forms a minimal superkey, and either one can be chosen as the primary key.

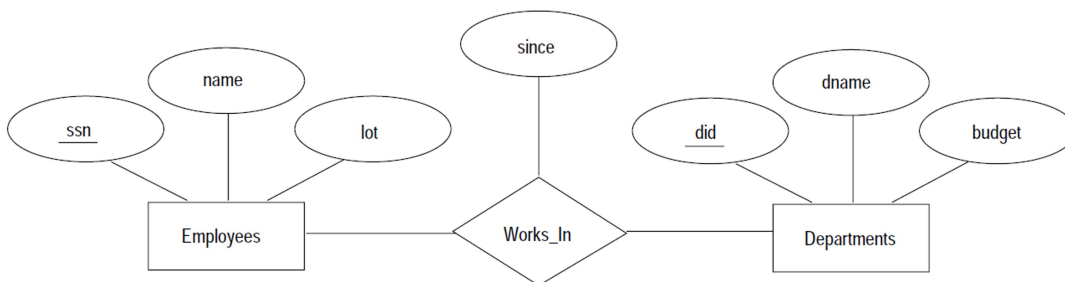


Figure The Works_In Relationship Set